DUY TÂN UNIVERSITY

International School

Group Project

CMU-SE 450 – C1SE.40

Project Proposal

Version 1.1

Date: September 6th, 2026

AI Voice Assistant Platform for Intelligent IoT Devices

Submitted by

Thinh, Do Ngoc

Long, Pham Hoang

Tri, Nguyen Huynh

Quang, Pham Dinh

Hoang,Tran Le

Approved by

MSc Hoang,Nguyen Thai

Proposal Review Panel Representative:

Name	Signature	Date

Group Project- Mentor:

Name	Signature	Date

PROJECT INFORMATION

Project acronym	AIVA-IoT		
Project Title	AI Voice Assistant Platform for Intelligent IoT Devices		
Start Date	6 Sep 2026	**End Date**	5 Dec 2026
Lead Institution	International School, Duy Tan University		
Project Mentor	MSc Hoang,Nguyen Thai		
Scrum master / Project Leader & contact details	Long, Pham Hoang Email: long290120005@gmail.com Tel: 0937806181		
Partner Organization	Duy Tan University		
Project Web URL			
Team members	Name	Email	Tel
1	Thinh, Do Ngoc	tungdongocdailoc@gmail.com	0774433725
2	Tri, Nguyen Huynh	tringuyen03112005@gmail.com	0762634005
3	Quang, Pham Dinh	pdinhquang05@gmail.com	0378410627
4	Long, Pham Hoang	long290120005@gmail.com	0937806181
5	Hoang, Tran Le	tranlehoang205@gmail.com	0898840964

DOCUMENT APPROVALS

The following signatures are required for approval of this document.

Hoang, Tran Le Student ID: 29211158794 Team leader	Signature	Date
Quang, Phạm Đình Student ID: 29211330920 *Team Member*	Signature	Date
Thinh, Do Ngoc Student ID:29211152892 *Team Member*	Signature	Date
Long, Phạm Hoàng Student ID: 29210254319 *Scrum master*	Signature	Date
Trí, Nguyễn Huỳnh Student ID:29211155138 *Team Member*	Signature	Date

REVISION HISTORY

Version	Date	Comments	Author
1.0	September 4rd, 2026	Initial Release	C1SE40
1.1	September 6th, 2026	Revised project proposal based on mentor feedback	C1SE40

TABLE OF CONTENT

1. Introduction 5
   1.1. Purpose of Document 5
   1.2. Project Goal 5
2. Problem Definition 6
   2.1. Users Need 7
   2.2. Solution 8
3. Current Status of Art 9
4. Engineering Approach 11
   4.1. System context diagram 11
   4.2. System context description 12
   4.3. Technical Constraints 13
5. Tasks and Deliverables 16
6. Project Management 17
   6.1. Cost/Budget for Project 18
   6.2. Tentative Schedule 18
      6.2.1. Master Plan 18
      6.2.3. Scrum Process 20
7. Project Constraints 21
8. Conclusion 23
9. References 24
10. Attachment 25

1. Introduction

1.1. Purpose of Document

This document is a Project Proposal that provides a comprehensive view of the End-to-End AI Voice Assistant System—a multi-module integrated solution that combines IoT embedded hardware based on the Xiaozhi ESP32 platform, a Realtime AI Server, and a centralized Management Web ecosystem.

- The document defines the needs of users and the needs of operating the system, in which the topic does not stop at customizing the firmware for a device but aims at the ability to centrally manage multiple users and multiple devices.
- Technical and business objectives: Build a user management mechanism, RBAC decentralization, API Key allocation and control according to quota, device pairing, telemetry monitoring, and device lifecycle management.
- System Architecture: Built-in Custom Firmware Xiaozhi ESP32, Realtime AI Communication Server and Web Admin/User Portal; the device communicates bi-directionally with the server via WebSocket for real-time data transmission.
- Technology solution: Use NestJS for the backend, Next.js for the frontend, MySQL for the database, ESP-IDF/C++ for the firmware, and the ASR/STT processing chain → LLM → TTS at the server. Use NestJS for the backend, Next.js for the frontend, MySQL for the database, and ESP-IDF/C++ for the firmware. The Realtime AI Server coordinates an ASR/STT → LLM → TTS processing pipeline. In Version 1, OpenAI API is used for LLM processing, while the Cloud ASR/STT and TTS providers will be selected during implementation based on Vietnamese language support, latency, reliability, API compatibility, and cost.

- AI Roadmap: Version 1 uses Cloud AI/OpenAI API Key to ensure feasibility and response speed; Version 2 gradually transitions to a self-hosted Local AI Model on a private GPU server to reduce costs, increase data control, and reduce dependence on external services.

1.2. Project Goal

The core goal of the project is to design and implement a complete Real-Time AI Virtual Assistant Ecosystem that seamlessly connects intelligent interactive hardware, AI dispatch servers, and centralized service administration systems.

1. Web Admin & User Portal: Build a Web platform using NestJS (Backend) and Next.js (Frontend), support login, RBAC permissions for Admin/User, account management, API Key allocation/revocation, and usage quota control

2. Device Management & Realtime Server: Allows pairing of Xiaozhi devices with accounts, device authentication, online/offline and telemetry monitoring, remote configuration, firmware/OTA management; while maintaining a two-way WebSocket connection for audio transmission and ASR/STT → LLM → TTS coordination.

3. Customized ESP32 Firmware: Customize the Xiaozhi firmware on ESP-IDF/C++ so that the device can securely connect to the project's own server, handle microphones/speakers, wake words, display status, and receive OTA configuration/updates.

4. Two-Phase AI Roadmap: Version 1 completes Web + Realtime Server + Firmware and uses using OpenAI API for LLM processing and Cloud AI services for ASR/STT and TTS. The specific Cloud ASR/STT and TTS providers will be selected during implementation based on Vietnamese language support, latency, reliability, API compatibility, and cost. Version 2 gradually replaces Cloud AI components with self-hosted Local AI Models on private GPU infrastructure, including local ASR, LLM, and TTS, aiming to reduce operating costs and improve data privacy.

2. Problem Definition

Today's AI tools offer strong conversational capabilities but still largely depend on phone/desktop applications or closed commercial ecosystems. This limits the ability to build a deeply customizable and administerable physical assistant device according to its unique needs.

If the topic only stops at the flash/custom firmware level for a Xiaozhi device, the system will be difficult to expand as the number of users and devices increases, and there is a lack of account administration mechanisms, decentralization, API Keys, quota, device pairing, telemetry, and remote firmware updates.

Technically, voice conversations require a low-latency communication channel between the device and the server. The server must maintain a real-time connection, receive the audio stream, and coordinate the entire ASR/STT → LLM → TTS pipeline before streaming feedback back to the device.

In terms of operation, the system needs a Web Admin/User Portal to manage users and centralized devices, protect API Keys, control usage limits, monitor online/offline status, telemetry and manage OTA firmware.

In addition, long-term reliance on Cloud AI increases costs with scale and causes data to go through external services. Therefore, the architecture needs to be modularized so that Version 1 uses OpenAI/Cloud APIs to prove feasibility, and then Version 2 can switch to the Local AI Model on its own GPU server.

2.1. Users Need

- Provides a natural, free, AI-supported voice interaction method via the Xiaozhi device.
- Ensures secure real-time or near-real-time conversations, handling audio data transmission and voice responses.
- Allows users to log in to the user portal, link devices to their accounts, and manage device usage permissions.
- Enables users to customize settings—such as response voices and system prompts—and view conversation history.
- Allows administrators to manage users, lock/unlock accounts, and assign permissions based on Role-Based Access Control (RBAC).

- Enables administrators to issue, revoke, and manage API keys, as well as configure rate limits and quotas to control resource usage and AI costs.

- Supports device system authentication, online/offline monitoring, heartbeat/remote status tracking, and operational status monitoring.Allows firmware version management and supports OTA updates for remote devices.

- Supports two-stage AI architecture: Version 1 uses OpenAI/Cloud API; Version 2 can switch to a self-hosted Local AI Model on the GPU server.

- Supports open architecture and modularization to expand the number of devices, change AI vendors, and deploy Local AI Models without having to redesign the entire firmware.

2.2. Solution

The team proposes to build a Comprehensive AI Virtual Assistant Ecosystem consisting of three pillar modules and a two-stage AI roadmap.

1. Edge Hardware – Xiaozhi ESP32: Custom firmware developed using C/C++ on ESP-IDF, performing Wi-Fi connections, device authentication, audio receiving/playback, wake word, status interface, and direct connection to the project's own server.

The firmware transmits the audio stream from the microphone to the server and receives real-time audio feedback via the WebSocket; and receives device configuration and OTA firmware updates when authorized.

2. Realtime AI Communication Server: The server maintains a two-way connection to Xiaozhi devices, manages sessions/heartbeats, and acts as an Orchestrator for the ASR/STT → LLM → TTS pipelines. Audio feedback is streamed back to the device in sections to reduce latency.

3. Management Web Platform: Backend NestJS provides RESTful API and WebSocket Gateway; Frontend Next.js provides Admin Dashboard and User Portal. The system supports Authentication/JWT, RBAC, User Management, API Key & Quota

Management, Device Assignment/Pairing, telemetry, AI configuration, conversation history, and OTA firmware management

Version 1 – Cloud API Foundation: Complete firmware, Realtime Server, NestJS Backend, and Next.js Portal; use OpenAI API Key/Cloud AI for AI processing to ensure stability and prove the entire realtime flow from device to server.

Version 2 – Local AI Integration: Gradually replace Cloud AI components with self-hosted models on private GPU servers, which can include local ASR, open-source LLM, and local TTS; implement latency, resource and cost benchmarks between Cloud and Local.

Version 1 Main Processing Flow:

User Voice → Xiaozhi ESP32 → WebSocket → Realtime AI Server → ASR/STT → OpenAI LLM → TTS → Realtime AI Server → Xiaozhi ESP32 → Voice Response

Admin Stream: Admin/User → Next.js Portal → NestJS REST API → MySQL; NestJS also coordinates with Realtime Server to manage device authentication, pairing, API Key/quota, telemetry, configuration, and OTA. The architecture refers to Xiaozhi's open server organization but is implemented according to the scope and technology of the project.

3. Current Status of Art

Feature	ChatGPT Voice	Xiaozhi AI Assistant	Proposed System
Voice Interaction	✓	✓	✓
Speech Recognition (STT)	✓	✓	✓
AI Conversation	✓	✓	✓

Text-to-Speech (TTS)	✓	✓	✓
Real-time Response	✓	✓	✓
Continuous Conversation	✓	✓	✓
Hardware Device Integration	—	✓	✓
Custom AI Model / API	—	✓	✓
Customizable Hardware	—	✓	✓
System Customization	—	✓	✓
Open-source / Extensible	—	✓	✓
Internet-based AI	✓	✓	✓

Existing voice assistants provide speech-based AI interaction, but the proposed system focuses on an open, manageable end-to-end ecosystem rather than only a voice feature or a customized single device.

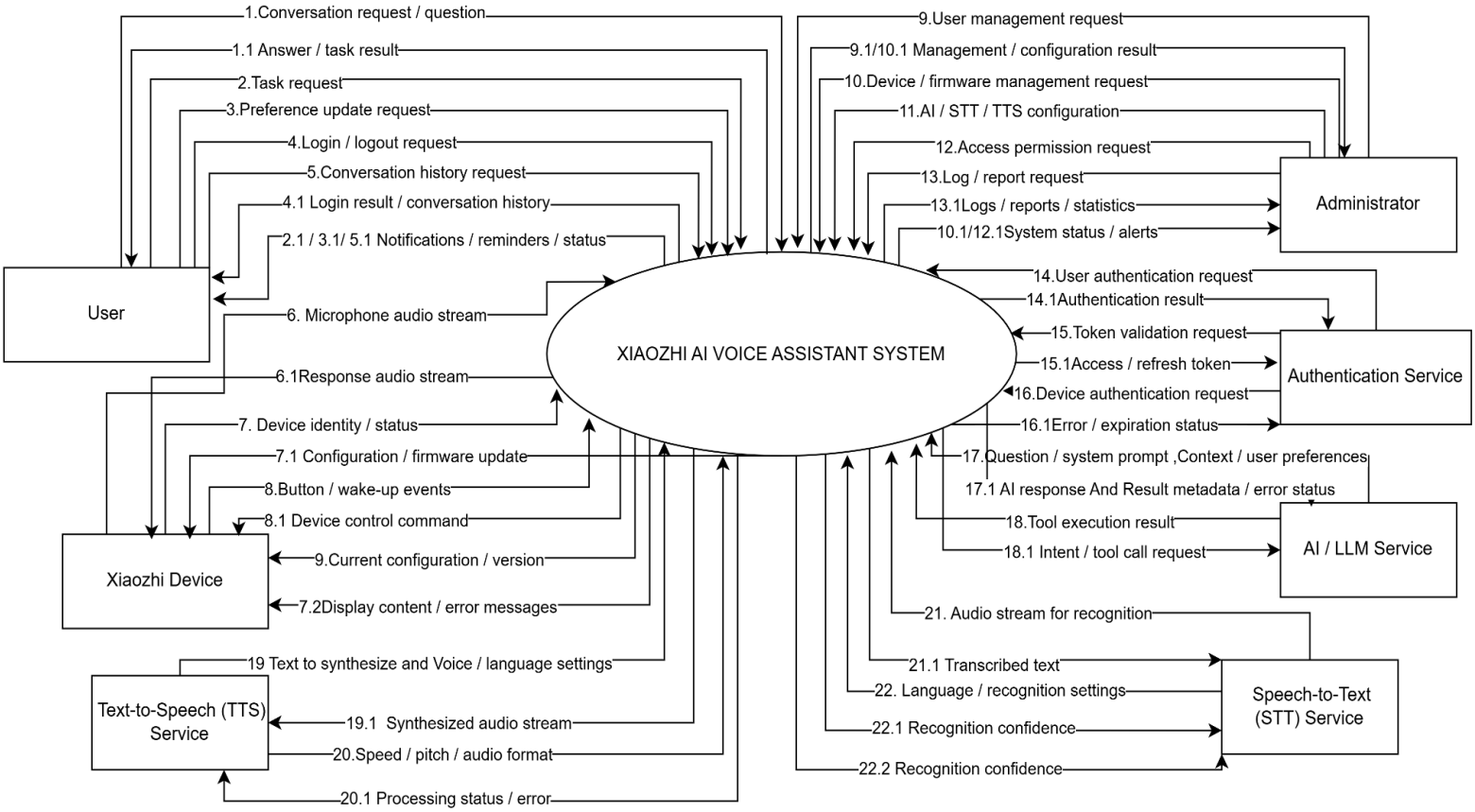
The proposed system differentiates itself by providing:

- Customized Xiaozhi ESP32 firmware connected to the project's own server.

- Low-latency duplex WebSocket communication for real-time audio streaming and device sessions.
- A Realtime AI Server that orchestrates ASR/STT → LLM → TTS and streams responses back to devices.
- A NestJS backend and Next.js Admin/User Portal instead of a device-only solution.
- Centralized authentication and RBAC for administrators and users.
- API Key lifecycle management with rate limiting/quota control.
- Device registration, assignment/pairing, online/offline status, heartbeat and telemetry monitoring.
- Remote AI configuration and OTA firmware management.
- A two-phase AI roadmap: Version 1 uses OpenAI/Cloud APIs; Version 2 migrates to self-hosted Local AI Models on a GPU server.
- Agile Scrum for iterative delivery across firmware, server, web platform and system integration.
- Software Quality Management covering functional testing, integration testing, latency, reliability and defect tracking.

4. Engineering Approach

4.1. System context diagram



4.2. System context description

The Xiaozhi AI Voice Assistant System is an end-to-end ecosystem consisting of customized Xiaozhi ESP32 firmware, a Realtime AI Communication Server, a NestJS management backend, a Next.js Admin/User Portal, MySQL, and integrated AI services.

The Xiaozhi device maintains a duplex WebSocket connection with the Realtime AI Server for audio streaming, device authentication, heartbeat and status exchange. The server orchestrates ASR/STT → LLM → TTS. The NestJS backend exposes RESTful APIs for the Next.js portal and manages users, RBAC, API Keys/quotas, device pairing, telemetry, AI configuration, conversation history and OTA firmware.

1. User: Log in to the User Portal, pair devices, interact with voice, view device/conversation history, and change permissioned configurations such as voice or system prompts.
2. Xiaozhi Device: Send audio stream, Device ID/token, heartbeat, and telemetry; receive audio feedback, configuration, and OTA information from the system through a secure communication channel.
3. ASR/STT service: Version 1 uses a Cloud ASR/STT provider to convert audio received from the Xiaozhi device into text. The specific provider will be selected during implementation based on Vietnamese language support, recognition accuracy, latency, reliability, API compatibility, and cost. Version 2 can switch to a self-hosted local ASR model running on a private GPU server.
4. TTS service: Version 1 uses a Cloud TTS provider to convert AI-generated text responses into speech before streaming audio back to the Xiaozhi device. The specific provider will be selected during implementation based on Vietnamese voice quality, latency, reliability, API compatibility, and cost. Version 2 can replace the Cloud TTS service with a self-hosted local TTS model.
5. AI/LLM: Version 1 uses OpenAI API Keys that are centrally managed and quota controlled. Version 2 gradually transitions to Local/Open-source LLMs that are self-hosted on private GPU servers.
6. Admin: Manages User/RBAC, API Key/quota, devices, pairing, telemetry, firmware/OTA, AI configuration, and operational reports/status.
7. Authentication Service: Authenticate the Admin/User with JWT and authenticate the device with the appropriate Device ID/token/API Key before allowing access to the system's resources.

4.3. Technical Constraints

Technical to develop:

- Programming Languages — C/C++ (Xiaozhi firmware), TypeScript (Backend and Frontend).

- Frameworks and Libraries — ESP-IDF, NestJS (Backend), Next.js (Frontend), React, Axios.

- Communication & Security — RESTful API, WebSocket, JWT Authentication, RBAC, rate limiting/quota, OTA Firmware Update.
- AI Roadmap — Version 1: OpenAI API for LLM processing, combined with Cloud ASR/STT and TTS services. The specific ASR/STT and TTS providers will be selected during implementation based on Vietnamese language support, latency, reliability, API compatibility, and cost. Version 2: Self-hosted Local AI Models for ASR, LLM, and TTS on a dedicated private GPU server.
- Hardware: Xiaozhi ESP32 device, microphone, speaker — Xiaozhi ESP32 device, microphone, speaker
- Database: MySQL
- Version Control: Git, GitHub.
- Team Management: Zalo, Google Drive.
- Development Tools: Visual Studio Code, ESP-IDF , Postman.

Environments:

- Development Environment: Visual Studio Code, ESP-IDF, Node.js, TypeScript.
- Firmware Development Environment: ESP-IDF, C/C++, ESP32 toolchain.
- Backend Environment: Node.js, TypeScript, NestJS, RESTful API, WebSocket Gateway, JWT/RBAC.
- Frontend Environment: Next.js, React, TypeScript, HTML, CSS.
- Database Environment: MySQL
- AI Environment: Version 1 uses OpenAI/Cloud APIs through centrally managed API Keys; Version 2 uses self-hosted ASR/LLM/TTS models on a GPU server.
- Testing Environment: Xiaozhi ESP32 device, microphone, speaker, local network, Postman.

- Version Control Environment: Git and GitHub.
- Team Collaboration Environment:, Zalo, Google Drive.
- Deployment Environment: NestJS backend and Realtime AI Server deployed on a server/cloud host with Internet access; Next.js Web Portal; MySQL database. Version 2 requires a stronger GPU server for Local AI Models.

Other Constraints:

Human resources: Group of 5 members, division of work by firmware modules, Realtime AI Server, NestJS Backend, Next.js Frontend, AI Integration, Testing and Documentation.

Budget: The project has a limited budget, prioritizing the use of open-source software and existing Xiaozhi hardware.

Time: The project must be completed within the implementation period of the project, which is expected to take about 3 months.

Internet connection: Version 1 requires a stable Internet for the device to connect to the Realtime Server and the server to call Cloud AI APIs; WebSockets need to be maintained stable to ensure a conversational experience.

Privacy and Security: Protect accounts, JWTs, Device Tokens, API Keys, quota, voice data, conversation history, and telemetry; apply RBAC and restrict access by role.

Device compatibility: The customized firmware must be compatible with the Xiaozhi ESP32 board, microphone, speakers, buttons, screen, and memory capacity of the device.

Dependence on external services: Version 1 depends on OpenAI/Cloud AI in terms of availability, latency, and cost; this is the reason why Version 2 is oriented towards switching to the Local AI Model.

Enhanced capabilities/capabilities in Version 2 or expansion:

- Complete local ASR/LLM/TTS deployment on private GPU infrastructure.
- Benchmark and optimize resources between Cloud AI and Local AI.
- Advanced smart home automation.
- Extend the Local AI Model and optimize inference for multiple devices simultaneously.
- Support more hardware platforms in addition to Xiaozhi ESP32.
- Telemetry analysis and advanced recommendations.

5. Tasks and Deliverables

No.	Task name	Description
1.	**Start-up**	
1.1	Project Kick-off Meeting	Meet with the development team and stakeholders to identify the project's objectives, scope, requirements, expected outcomes, and initial technical direction for the AI voice assistant using the Xiaozhi device.
1.2	Discuss Project Requirements	Discuss and clarify the project's main ideas, user requirements, system functions, communication flow, and expected interaction between the user, Xiaozhi device, and AI assistant.
1.3	Create Project Documents	Prepare and submit the necessary project artifacts, including Project Proposal, User Stories, Product Backlog, Project Plan, system requirements, and initial system design.
2.	**Development**	
2.1	Sprint Planning	Conduct Sprint Planning to define the objectives, tasks, priorities, and expected deliverables for each sprint based on the Product Backlog.

2.2	Sprint 1 – Platform Foundation & Firmware	Set up NestJS backend, Next.js frontend and MySQL; implement authentication/RBAC foundation; customize Xiaozhi ESP32 firmware and establish secure WebSocket connectivity with the Realtime AI Server.
2.3	Sprint 2 – Realtime AI & Management	Implement ASR/STT → OpenAI LLM → TTS orchestration, audio streaming, user/device management, pairing, API Key lifecycle and quota, online/offline status, heartbeat/telemetry, conversation history and AI configuration.
2.4	Sprint 3 – Portal, OTA & Integration	Complete Admin Dashboard and User Portal, OTA firmware management, end-to-end integration, security/error handling, latency and reliability testing, and finalize Version 1. Prepare the architecture and benchmark plan for Version 2 Local AI migration.
3.	**Project Meeting**	Conduct an internal project meeting to review the completed functions, identify remaining issues, perform final testing, and prepare the system and presentation materials for the stakeholder/customer.
4.	**Final Release**	Release Version 1 with customized Xiaozhi firmware, Realtime AI Server, NestJS backend, Next.js Admin/User Portal, MySQL, OpenAI/Cloud AI integration, device management, API Key/quota, telemetry and OTA. Deliver source code, deployment/configuration instructions, test results and project documents; document Version 2 Local AI migration plan.

6. Project Management

6.1. Cost/Budget for Project

Full Name	Role	Salary Rate (USD/hour)
Hoàng, Trần Lê	Team Leader	3
Trí, Nguyễn Huỳnh	Team Member	3
Quang, Phạm Đình	Team Member	3
Long, Phạm Hoàng	Scrum Master	3
Thịnh, Đỗ Ngọc	Team Member	3

Table 1. Cost person/hours

No	Criteria	Price	Total (USD)
1	Working hours	3	4050
2	Other cost	100	500
			4550

Table 2. Total cost estimation

Description	Amount	Unit.
Number of members	5	Person
Number of working hours per day	3	Hours
The cost per hour per member	3	USD
The number of working days	90	Days

Table 3. Description

- The explanation for the table

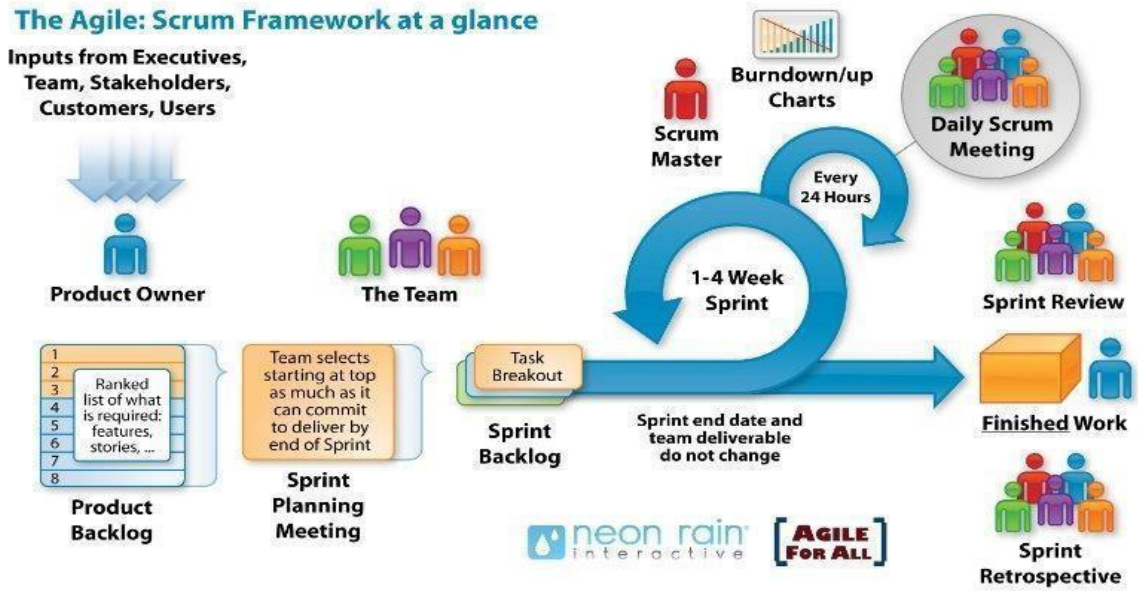
- Amount of working hours = 5 members * 3 hours * 90 days
- Other cost = 5 members * 100 USD

6.2. Tentative Schedule

6.2.1. Master Plan

No.	Task Name	Duration	Start	Finish
1	**Initial**	**7 days**	**06-Sep-2026**	**12-Sep-2026**
1.1	Gathering Requirements	2 days	06-Sep-2026	07-Sep-2026
1.2	Create Proposal Document	5 days	08-Sep-2026	12-Sep-2026
2	**Start Up**	**7 days**	**13-Sep-2026**	**19-Sep-2026**
2.1	Project Kick-off Meeting	1 day	13-Sep-2026	13-Sep-2026
2.2	Create Project Documents	6 days	14-Sep-2026	19-Sep-2026
3	**Development**	**75 days**	**20-Sep-2026**	**03-Dec-2026**
3.1	Sprint 1	28 days	20-Sep-2026	17-Oct-2026
3.2	Sprint 2	23 days	18-Oct-2026	09-Nov-2026
3.3	Sprint 3	24 days	10-Nov-2026	03-Dec-2026
4	**Project Retrospective Meeting**	**1 day**	**04-Dec-2026**	**04-Dec-2026**
5	**Final Release**	**1 day**	**05-Dec-2026**	**05-Dec-2026**

6.2.3. Scrum Process



- Scrum is an iterative and incremental agile software development framework for managing software projects and product or application development.
- Scrum focuses on project management institutions where it is difficult to plan ahead.
- Mechanisms of empirical process control, where feedback loops that constitute the core management technique is used as opposed to traditional command-and -control management.
- Its approach to planning and managing projects is by bringing decision-making authority to the level of operation properties and certainties.
- Benefit of the methodology:
  - Project can respond easily to change.
  - Problems are identified early.
  - Customers get the most beneficial work first.
  - Work done will better meet the customer's needs.
  - Improved productivity.
  - Ability to maintain a predictable schedule for delivery.

7. Project Constraints

Constraint	Description	Guidelines for Acceptance
Economic	● The budget under section 6.1 includes $4,050 in labor (5 members × 3 hours/day × 90 days of estimated × $3/hour) and $500 for components, prototypes, servers, and APR/LLM/TTS APIS; a total of $4,550. ● Prioritize core conversational functionality and limit API usage. The freemium model is only the direction of commercialization in the future.	● There is a cost tracking table; the components, operations and APIs do not exceed 500 USD, and the total conversion cost does not exceed 4,550 USD. Changes that exceed the estimate must be reviewed and approved before implementation.
Environmental	● The prototype uses a board, microphone, speaker, and device case, so e-waste can be generated. Prioritize reuse and replacement of each component. ● Evaluate power in standby and conversation; select a power-saving mode in accordance with the wake-up mechanism, avoid unnecessary audio transmission or API calls.	● There is a component catalog, a failure component collection plan, and power measurement results in two states. The power-saving configuration still meets the selected trigger mechanism; does not declare the savings when not measured.
Ethical	● Clearly notifies when the device records audio and when data is sent to an external AI service. Interactive recording only has been enabled; raw audio is not saved by default. ● If the conversation history is saved, consent, access, and a deletion mechanism are required. Protect the API key, record the source and license of the firmware and libraries used.	● Test recording status, access permissions, and delete history if applicable. Don't leave API keys in public source code or logs. Have a data flow description and open source component catalog with a license.

Public Health, Safety, and Welfare	● Use a power supply suitable for the component, shield the contacts, and secure the extension cord. If using a battery, select the appropriate charging and protection circuitry. ● Allows volume adjustment, feedback pause, and recording off. Notifies that the AI answer may be wrong; the device is not designed as an emergency system or hazardous task control.	● Check the assembly, power supply, and temperature during commissioning according to component specifications. The operations to stop playback, adjust the volume, and turn off the recording work properly; the manual states the usage limits and how to disconnect the power.
Social and Global	● Voice communication aims to reduce input manipulation in learning and information lookup. The trigger button supports when the wake-up keyword works inappropriately. ● Usability also depends on the Internet, noise, and AI services. When the connection is lost or the service fails, the device needs to report the status and allow the retry.	● Q&A testing with users with different levels of technology familiarity and recording difficulties. Testing of trigger buttons, network outages, and service errors; the device clearly reports errors, does not replay old answers as new results.
Cultural	● Prioritize Vietnamese conversation, polite addresses, and contextually appropriate responses. Regional accents, proper names, and loanwords can affect identification. ● When the request is not understood, the system needs to ask the user to restate or clarify. Support for other languages is the direction of expansion, depending on the selected ASR/TTS service.	● Develop a set of Vietnamese test questions with regional accents, proper names and common communication situations. Report identification errors, intelligibility of the voice and unclear handling of requests; record unsupported cases.

Sustainability	● The group of 5 members will implement from 06/09 to 05/12/2026 according to the three sprints in section 6.2. Prioritize the completion of the prototype; Smart Home and expansion services depend on the remaining resources. ● Firmware, backend, and ASR/LLM/TTS connectors for easy maintenance. Manage source code versions, configurations, and documentation to support service changes or transfers.	● Deliver source code, firmware build/load instructions, configuration, operation, and test reports. Run at least 20 consecutive Q&A, recording delays and errors; test recovery after a network outage. Complete the release and restriction list before the deadline.

8. Conclusion

The project aims for a complete AI assistant ecosystem consisting of Xiaozhi ESP32 Custom Firmware, Realtime AI Communication Server, and Web Admin/User Portal, rather than just customizing the firmware for a single device.

Version 1 uses NestJS for the Backend, Next.js for the Frontend, and MySQL; the server maintains WebSockets with the device, coordinates ASR/STT → OpenAI LLM → TTS, and provides administrative functions including Authentication/JWT, RBAC, User Management, API Key/quota, Device Pairing, telemetry, AI configuration, conversation history, and OTA.

Version 2 aims to gradually replace the Cloud/OpenAI API with a self-hosted Local AI Model on its own GPU server. The transition is evaluated by latency, hardware resources, operating costs, and data privacy; the modular architecture helps to change the AI layer without having to completely redesign the firmware and Web Platform.

The expected result is a platform capable of centralized management, multi-device expansion, and continued development according to the Cloud-to-Local AI roadmap, while helping the team apply its knowledge of embedded systems, real-time communications, web development, AI integration, security, and software project management.

9. References

[1]. Software Development Standards for the Guidance and Control Software Project: *https://sw-eng.larc.nasa.gov*

[2]. General Software Coding Standards and Guidelines: *https://www.nws.noaa.gov/oh/hrl/developers_docs/General_Software_Standards.pdf*

[3]. Scrum and best practices*: https://docs.microsoft.com/en-us/azure/devops/boards/sprints/best-practices-scrum?view=azure-devops*

[4]. The Scrum Guide: *https://www.scrum.org/resources/scrum-guide*

[5]. The ISO/IEC & IEEE/EIA Standard 12207, IEEE standards: IEEE-829 [3], IEEE-1008 [5], IEEE-1012

[6]. NestJS Documentation: https://docs.nestjs.com/

[7]. Next.js Documentation: https://nextjs.org/docs

[8]. Xiaozhi ESP32 Server (open-source): https://github.com/xinnan-tech/xiaozhi-esp32-server

[9]. OpenAI API Documentation: https://platform.openai.com/docs

10. Attachment